Contents

1 Template

```cpp
#include <bits/stdc++.h>

using namespace std;

#define all(v) v.begin(),v.end()
#define rall(v) v.rbegin(),v.rend()
#define pb push_back
#define sz(a) int(a.size())
#define F first
#define S second
typedef long long ll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef pair<int,int> pii;
typedef pair<ll,ll> pll;
const int MOD = 1e9+7;
const int INF = INT_MAX;
const ll INF64 = LLONG_MAX;
const long double EPS = 1e-9;
const long double PI = acosl(-1.0L);
void setIO(string p){
  freopen((p + ".in").c_str(), "r", stdin);
  freopen((p + ".out").c_str(), "w", stdout);
}

void solve(){

}


int main(){
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout.tie(0);
  int tc = 1;
  //cin >> tc;
  for (int t = 1; t <= tc; t++)solve();
}
```

2 Data structures

2.1 Segment Tree

```cpp
struct Node{
  ll val = 0;
};
Node operator+(Node a,Node b){return {a.val+b.val};}
class Segment_Tree{
public:
  int n;
  vector<Node> tree;

  Segment_Tree(int x = 1e5+10){
    n = x;
    tree.resize(2*n+1);
  }

  void update(int p, ll val){
    for(tree[p+=n] = {val};p>1;p>>=1)
      tree[p>>1] = tree[p]+tree[p^1];
    //for (tree[p+=n] = {val};p>>= 1;)t[p] = t[p<<1]+t[p<<1|1);
  }
  ll query(int l,int r){
    Node s;
    for(l+=n,r+=n ; l<r ; l>>=1 , r>>=1){
      if(l&1)s = s+ tree[l++];
      if(r&1)s = s+ tree[--r];
    }
    return s.val;
  }
};
```

2.2 Merge Sort Tree

```
struct Node{
  vi val;

  int search(int x){
    int l = 0, r = sz(val)-1;
    while(l<=r){
      int m = l+(r-l)/2;
      if(val[m] < x)l =m+1;
      else r = m-1;
    }
    return sz(val)-l;
  }
};

Node operator+(Node a,Node b){
  int i=0,j=0;
  Node aux;
  while (i< sz(a.val) && j<sz(b.val)){
    if (a.val[i]< b.val[j] ) aux.val.pb(a.val[i++]);
    else aux.val.pb(b.val[j++]);
  }
  while (i<sz(a.val)) aux.val.pb(a.val[i++]);
  while (j<sz(b.val)) aux.val.pb(b.val[j++]);
  return aux;
}
//Call segment Tree
```

2.3 Fenwick Tree

```
struct Node{
  ll n = 0;
  Node operator+(Node b){return {n+b.n};}
};

class Fenwick_Tree{
public:
  vector<Node> tree;
  int n;
  Fenwick_Tree(int x){
    n = x+1;tree.resize(n);
  }

  ll sum(int r){
    r++;
    Node ans = {0};
    while(r){
      ans = ans + tree[r];
      r -= r&-r;
    }
    return ans.n;
  }

  void update(int id, Node val){
    id++;
    while(id < n){
      tree[id]  = tree[id]+val;
      id += id&-id;
    }
  }
};
```

2.4 Sparse Table

```
struct Node{
  int val = 0;
  Node operator+(Node b){return {__gcd(val,b.val)};}
};

class SparseTable{
  public:
  vector<vector<Node>> sparse;

  SparseTable(int n=2e5+10){
    sparse.assign(n , vector<Node> (log2(n)+1));
  }

  void build(const vi &a){
    for(int i=0;i<sz(a);i++)
      sparse[i][0] = {a[i]};

    for(int j=1;(1<<j)<=sz(a);j++)
      for(int i=0;i+(1<<j)-1 <sz(a);i++)
        sparse[i][j]= sparse[i][j-1]+sparse[i+(1<<(j-1))][j-1];
  }

  int query(int l, int r){
    int len = r-l+1;
    int k = log2(len);
    return (sparse[l][k]+sparse[r-(1<<k)+1][k]).val;
  }

};
```

2.5 Disjoint Set Union

```
class DSU{
  private:
  vi parent,rank,size;
  int c;
  public:
  int mx = 0;
  DSU(int n):parent(n+1),rank(n+1,0),size(n+1,1),c(n){
    iota(all(parent),0);
  }

  int find(int u){return parent[u] == u?u:parent[u] = find(parent[u]) ;}

  bool same(int u,int v){return find(u)==find(v);}

  int get_size(int u){return size[find(u)];}

  int count(){return c;}

  void merge(int u,int v){
    u = find(u);
    v = find(v);
    if(u!=v){
      c--;
      if(rank[u] > rank[v])swap(u,v);
      parent[u] = v;
      size[v] += size[u];
      if(rank[u] == rank[v])rank[v]++;
      mx = max(mx,size[v]);
    }
  }
};
```

2.6 Lazy SegTree

```
struct Segment_Tree{

  int n;
  vector<Node> tree;
  vector<Lazy> lazy;

  Segment_Tree(int x = 1e5+10){
    n = x;
    tree.resize(4*n);
    lazy.resize(4*n);
  }

  void push_set(int v,int l,int r){
    if(!lazy[v].has_set)return;
    tree[v].val = lazy[v].val*(r-l+1);

    if(l!=r){
      lazy[v<<1] = lazy[v];
      lazy[(v<<1)|1] = lazy[v];
    }
    lazy[v].has_set = 0;
    lazy[v].val = 0;
  }

  void push_sum(int v,int l,int r){
    if(!lazy[v].has_sum)return;
    tree[v].val = tree[v].val+lazy[v].val*(r-l+1);
    if(l!=r){
      lazy[v<<1].val = lazy[v<<1].val + lazy[v].val;
      lazy[(v<<1)|1].val = lazy[(v<<1)|1].val + lazy[v].val;
      if(!lazy[v<<1].has_set)lazy[v<<1].has_sum=1;
      if(!lazy[(v<<1)|1].has_set)lazy[(v<<1)|1].has_sum=1;
    }
    lazy[v].has_sum = 0;
    lazy[v].val= 0;
  }

  void update(int pos,ll val,int v,int tl, int tr){
    if(tl == tr)tree[v] = {val};
    else{
      int tm = tl+(tr-tl)/2;
      if(pos <= tm)update(pos,val,v<<1,tl,tm);
      else update(pos,val,(v<<1)+1,tm+1,tr);
      tree[v] = tree[v<<1]+tree[(v<<1)+1];
    }
  }

  void update_set(int v,int tl,int tr,int l,int r,ll val){
    push_set(v,tl,tr);
    push_sum(v,tl,tr);
    if(l>tr||r<tl)return;
    if(tl>= l && tr <=r){
      lazy[v].has_set=1;
      lazy[v].has_sum=0;
      lazy[v].val=val;
      push_set(v,tl,tr);
      return;
    }
    int tm = tl+(tr-tl)/2;
    update_set(v<<1,tl,tm,l,r,val);
    update_set((v<<1)|1,tm+1,tr,l,r,val);
    tree[v] = tree[v<<1]+tree[(v<<1)|1];
  }

  void update_sum(int v,int tl,int tr,int l,int r,ll val){
    push_set(v,tl,tr);
    push_sum(v,tl,tr);
    if(l>tr||r<tl)return;
    if(tl>= l && tr <=r){
      lazy[v].has_sum=1;
      lazy[v].val += val;
      push_sum(v,tl,tr);
      return;
    }
    int tm = tl+(tr-tl)/2;
    update_sum(v<<1,tl,tm,l,r,val);
    update_sum((v<<1)|1,tm+1,tr,l,r,val);
    tree[v] = tree[v<<1]+tree[(v<<1)|1];
  }

  Node query(int v ,int tl ,int tr ,int l ,int r){
    push_set(v,tl,tr);
    push_sum(v,tl,tr);
    if(l>tr||r<tl)return {0};
```

```
    if(tl>= l && tr <= r )
      return tree[v];
    int tm = tl+(tr-tl)/2;
    return query(v<<1,tl,tm,l,r)+query((v<<1)|1,tm+1,tr,l,r);
  }

};
```

2.7 Implicit Treap

```
struct node {
    int val, sz, prior, lazy, sum, mx, mn, repl;
    bool repl_flag, rev;
    node *l, *r, *par;
    node() {
      lazy = 0;
      rev = 0;
      sum = 0;
      val = 0;
      sz = 0;
      mx = 0;
      mn = 0;
      repl = 0;
      repl_flag = 0;
      prior = 0;
      l = NULL;
      r = NULL;
      par = NULL;
    }
    node(int _val) {
      val = _val;
      sum = _val;
      mx = _val;
      mn = _val;
      repl = 0;
      repl_flag = 0;
      rev = 0;
      lazy = 0;
      sz = 1;
      prior = rnd();
      l = NULL;
      r = NULL;
      par = NULL;
    }
};

struct treap {
  typedef node* pnode;
  pnode root;
  map<int, pnode> position;//positions of all the values
  //clearing the treap
  void clear() {
    root = NULL;
    position.clear();
  }

  treap() {
    clear();
  }

  int size(pnode t) {
    return t ? t->sz : 0;
  }
  void update_size(pnode &t) {
    if(t) t->sz = size(t->l) + size(t->r) + 1;
  }

  void update_parent(pnode &t) {
    if(!t) return;
    if(t->l) t->l->par = t;
    if(t->r) t->r->par = t;
  }
  //add operation
  void lazy_sum_upd(pnode &t) {
    if( !t or !t->lazy )    return;
    t->sum += t->lazy * size(t);
    t->val += t->lazy;
    t->mx += t->lazy;
    t->mn += t->lazy;
    if( t->l )  t->l->lazy += t->lazy;
    if( t->r )  t->r->lazy += t->lazy;
    t->lazy = 0;
  }
  //replace update
  void lazy_repl_upd(pnode &t) {
    if( !t or !t->repl_flag )   return;
```

```
  t->val = t->mx = t->mn = t->repl;
  t->sum = t->val * size(t);
  if( t->l ) {
    t->l->repl = t->repl;
    t->l->repl_flag = true;
  }
  if( t->r ) {
    t->r->repl = t->repl;
    t->r->repl_flag = true;
  }
  t->repl_flag = false;
  t->repl = 0;
}
//reverse update
void lazy_rev_upd(pnode &t) {
  if( !t or !t->rev ) return;
  t->rev = false;
  swap(t->l, t->r);
  if( t->l )  t->l->rev ^= true;
  if( t->r )  t->r->rev ^= true;
}
//reset the value of current node assuming it now
//represents a single element of the array
void reset(pnode &t) {
  if(!t) return;
  t->sum = t->val;
  t->mx = t->val;
  t->mn = t->val;
}
//combine node l and r to form t by updating corresponding queries
void combine(pnode &t, pnode l, pnode r) {
  if(!l) {
    t = r;
    return;
  }
  if(!r) {
    t = l;
    return;
  }
  //Beware!!!Here t can be equal to l or r anytime
  //i.e. t and (l or r) is representing same node
  //so operation is needed to be done carefully
  //e.g. if t and r are same then after t->sum=l->sum+r->sum operation
      ,
  //r->sum will be the same as t->sum
  //so BE CAREFUL
  t->sum = l->sum + r->sum;
  t->mx = max(l->mx, r->mx);
  t->mn = min(l->mn, r->mn);
}
//perform all operations
void operation(pnode &t) {
  if( !t )    return;
  reset(t);
  lazy_rev_upd(t->l);
  lazy_rev_upd(t->r);
  lazy_repl_upd(t->l);
  lazy_repl_upd(t->r);
  lazy_sum_upd(t->l);
  lazy_sum_upd(t->r);
  combine(t, t->l, t);
  combine(t, t, t->r);
}
//split node t in l and r by key k
//so first k+1 elements(0,1,2,...k) of the array from node t
//will be split in left node and rest will be in right node
void split(pnode t, pnode &l, pnode &r, int k, int add = 0) {
  if(t == null) {
    l = null;
    r = null;
    return;
  }
  lazy_rev_upd(t);
  lazy_repl_upd(t);
  lazy_sum_upd(t);
  int idx = add + size(t->l);
  if(t->l) t->l->par = null;
  if(t->r) t->r->par = null;
  if(idx <= k)
    split(t->r, t->r, r, k, idx + 1), l = t;
  else
    split(t->l, l, t->l, k, add), r = t;

  update_parent(t);
  update_size(t);
  operation(t);
```

```
}
//merge node l with r in t
void merge(pnode &t, pnode l, pnode r) {
  lazy_rev_upd(l);
  lazy_rev_upd(r);
  lazy_repl_upd(l);
  lazy_repl_upd(r);
  lazy_sum_upd(l);
  lazy_sum_upd(r);
  if(!l) {
    t = r;
    return;
  }
  if(!r) {
    t = l;
    return;
  }

  if(l->prior > r->prior)
    merge(l->r, l->r, r), t = l;
  else
    merge(r->l, l, r->l), t = r;

  update_parent(t);
  update_size(t);
  operation(t);
}
//insert val in position a[pos]
//so all previous values from pos to last will be right shifted
void insert(int pos, int val) {
  if(root == NULL) {
    pnode to_add = new node(val);
    root = to_add;
    position[val] = root;
    return;
  }

  pnode l, r, mid;
  mid = new node(val);
  position[val] = mid;

  split(root, l, r, pos - 1);
  merge(l, l, mid);
  merge(root, l, r);

  /*pnode x = new Node(val),merge(root,root,x);*/
}
//erase from qL to qR indexes
//so all previous indexes from qR+1 to last will be left shifted qR-qL +1 times
void erase(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  merge(root, l, r);
}
//returns answer for corresponding types of query
int query(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);

  int answer = mid->sum;
  merge(r, mid, r);
  merge(root, l, r);

  return answer;
}
//add val in all the values from a[qL] to a[qR] positions
void update(int qL, int qR, int val) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  lazy_repl_upd(mid);
  mid->lazy += val;

  merge(r, mid, r);
  merge(root, l, r);
}
//reverse all the values from qL to qR
void reverse(int qL, int qR) {
  pnode l, r, mid;

```

```
  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);

  if(mid)mid->rev ^= 1;
  merge(r, mid, r);
  merge(root, l, r);
}
//replace all the values from a[qL] to a[qR] by v
void replace(int qL, int qR, int v) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  lazy_sum_upd(mid);
  mid->repl_flag = 1;
  mid->repl = v;
  merge(r, mid, r);
  merge(root, l, r);
}
//it will cyclic right shift the array k times
//so for k=1, a[qL]=a[qR] and all positions from ql+1 to qR will
//have values from previous a[qL] to a[qR-1]
//if you make left_shift=1 then it will to the opposite
void cyclic_shift(int qL, int qR, int k, bool left_shift = 0) {
  if(qL == qR) return;
  k %= (qR - qL + 1);

  pnode l, r, mid, fh, sh;
  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);

  if(left_shift == 0) split(mid, fh, sh, (qR - qL + 1) - k - 1);
  else split(mid, fh, sh, k - 1);
  merge(mid, sh, fh);

  merge(r, mid, r);
  merge(root, l, r);
}
bool exist;
//returns index of node curr
int get_pos(pnode curr, pnode son = nullptr) {
  if(exist == 0) return 0;
  if(curr == NULL) {
    exist = 0;
    return 0;
  }
  if(!son) {
    if(curr == root) return size(curr->l);
    else return size(curr->l) + get_pos(curr->par, curr);
  }

  if(curr == root) {
    if(son == curr->l) return 0;
    else return size(curr->l) + 1;
  }

  if(curr->l == son) return get_pos(curr->par, curr);
  else return get_pos(curr->par, curr) + size(curr->l) + 1;
}
//returns index of the value
//if the value has multiple positions then it will
//return the last index where it was added last time
//returns -1 if it doesn't exist in the array
int get_pos(int value) {
  if(position.find(value) == position.end()) return -1;
  exist = 1;
  int x = get_pos(position[value]);
  if(exist == 0) return -1;
  else return x;
}
//returns value of index pos
int get_val(int pos) {
  return query(pos, pos);
}
//returns size of the treap
int size() {
  return size(root);
}
//inorder traversal to get indexes chronologically
void inorder(pnode cur) {
  if(cur == NULL) return;
  operation(cur);
  inorder(cur->l);
  cout << cur->val << ' ';
  inorder(cur->r);
}
```

```

  bool find(int val) {
    if(get_pos(val) == -1) return 0;
    else return 1;
  }
};

treap t;
//Beware!!!here treap is 0-indexed

int main() {
  int i, j, k, n, m, l, r, q;
  for(i = 0; i < 10; i++) t.insert(i, i * 10);
  t.cyclic_shift(4, 5, 1);
  t.update(2, 5, 1);
  t.replace(2, 5, 100);
  t.reverse(2, 9);
  t.replace(2, 5, 200);
  cout << t.query(0, 7) << nl;
  t.cyclic_shift(2, 3, 2, 1);
  cout << t.get_pos(20) << nl;
  t.erase(2, 2);
  cout << t.find(30) << nl;
  t.print_array();
  return 0;
}
```

2.8 Trie

```
struct Node{
  int v;
  int cnt = 0;
  Node* nxt[2];
  Node(int __v){
    v = __v;
    cnt = 0;
    nxt[0] = nxt[1] = NULL;
  }
};

struct Trie{
  typedef Node* pnode;
  pnode root;
  const int B = 30;

  void insert(int val){
    pnode cur = root;
    for(int i=B;i>=0;i--){
      bool b = val>>i&1;
      if(cur->nxt[b]==NULL)cur->nxt[b] = new Node(b);
      cur = cur->nxt[b];
      cur->cnt++;
    }
  }

  pnode erase(pnode cur, int val, int i) {
    if (cur == NULL)return NULL;
    cur->cnt--;
    if (i < 0) {
      if (cur->cnt == 0) {
        delete cur;
        return NULL;
      }
      return cur;
    }

    bool b = (val >> i) & 1;
    cur->nxt[b] = erase(cur->nxt[b], val, i - 1);

    if (cur->cnt == 0) {
      delete cur;
      return NULL;
    }

    return cur;
  }

  int query(int val){
    pnode cur = root;
    int ans = 0;
    for(int i = B;i>=0;i--){
      bool b = val>>i&1;
      if(cur->nxt[!b])cur = cur->nxt[!b],ans<<=1,ans++;
      else cur = cur->nxt[b],ans<<=1;
    }
    return ans;
```

```
  }
};
```

3 Graphs

3.1 DFS

```
void dfs(int u){
  vis[u] = 1;
  for(auto v:adj[u]){
    if(!vis[v])dfs(v);
  }
}
```

3.2 BFS

```
queue<int> q;
q.push(0);
while(!q.empty()){
  int u = q.front();
  q.pop();
  for(int v:adj[u]){
    if (!vis[v]) {
      vis[v] = 1;
      q.push(v);
    }
  }
}
```

3.3 Topological Sort

```
vi ts;
queue<int> q;
for (int i = 1; i < n+1; i++)
  if(!in[i])q.push(i);

while (!q.empty()) {
  int u=q.front();
  q.pop();
  ts.pb(u);
  for (int v : adj[u]) {
    in[v]--;
    if(!in[v])q.push(i);
  }
}

if(sz(ts)!=n)cout <<"IMPOSSIBLE";
```

3.4 Flood Fill

```
int dx[] = {1,1,0,-1,-1,-1,0,1};
int dx[] = {0,1,1,1,0,-1,-1,-1};

int floodfill(int x,int y,char color1,char color2){
  if(grid[x][y] == "outside")return 0;
  if(grid[x][y] != color1)return 0;
  int ans = 1;
  grid[x][y] = color2;
  for(int d=0;d<8;d++)ans += floodfill(x+dx[d],y+dy[d],color1,colr2);
  return ans;
}
```

3.5 MST

3.5.1 Kruskal

```
struct Edge{
  int u,v,w;
  bool operator < (Edge b){return w<b.w;}
};

//inside main

vector<Edge> edges;
sort(all(edges));

DSU dsu(vertices);

for(int i=0;i<E;i++){
  Edge front = edges[i];
  if(!dsu.same(front.u,front.v)){
    cost+= front.w;
    dsu.merge(front.u,front.v);
  }
}
```

3.5.2 Prims

```
vector<bool> vis(n);
priority_queue<pii, vector<pii>, greater<pii>> q;

auto process = [&](int u){
  vis[u] = 1;
  for(int i=0;i<sz(adj[u]);i++){
    pii v = adj[u][i];
    swap(v.S,v.F);
    q.push(v);
  }
}
process(0);

while(!q.empty()){
  auto [w,u] = q.top();q.pop();
  if(!vis[u]){cost += w;process(u);}
}
```

3.6 Dijkstra

3.6.1 Implementation

```
vll dist(n,INF64);
vector<bool> vis(n,0);
priority_queue<pll,vector<pll>,greater<pll>> pq;
pq.emplace(0,0);
dist[0] = 0;
vll cnt(n,0);//number of shortest paths
while(!pq.empty()){
  auto [d1,u] = pq.top();
  pq.pop();
  if(vis[u])continue;
  vis[u] = 1;
  for(auto [v,c]:adj[u]){
    if(dist[u]+c < dist[v]){
      dist[v] = dist[u]+c;
      pq.emplace(dist[v],v);
      cnt[v] = cnt[u];
    }else if(dist[u]+c == dist[v])(cnt[v]+=cnt[u])%=MOD;
  }
}
```

3.6.2 Space-State graph

```
vector<vll> dist(n,vll(1001,INF64));
vector<vector<bool>> vis(n,vector<bool> (1001,0));
typedef array<ll,3> T;
priority_queue<T ,vector<T>,greater<T>> pq;
pq.push({0,0,state[0]});
dist[0][state[0]]= 0;
while(!pq.empty()){
  auto [d,u,s] = pq.top();
  pq.pop();
  if(vis[u][s])continue;
  vis[u][s] = 1;
  for(auto [v,w]:adj[u]){
    ll c  = min(s,slow[v]);
    //Nuevo estad cuando llegue a V
    if(dist[u][s]+w*s<dist[v][c]){
      dist[v][c] = dist[u][s]+w*s;
      pq.push({dist[v][c],v,c});
    }
  }
}
```

3.7 Bellman-Ford

3.7.1 Path

```
vector<int> d(n, INF);
d[v] = 0;
vector<int> p(n, -1);

for (;;) {
bool any = false;
for (Edge e : edges)
    if (d[e.a] < INF)
  if (d[e.b] > d[e.a] + e.cost) {
      d[e.b] = d[e.a] + e.cost;
      p[e.b] = e.a;
      any = true;
  }
if (!any)
    break;
}
```

```cpp
if (d[t] == INF)
  cout << "No path from " << v << " to " << t << ".";
else {
  vector<int> path;
  for (int cur = t; cur != -1; cur = p[cur])path.push_back(cur);
  reverse(path.begin(), path.end());
}
```

3.7.2 Cycle

```cpp
vector<int> d(n, INF);
d[v] = 0;
vector<int> p(n, -1);
int x;
for (int i = 0; i < n; ++i) {
x = -1;
for (Edge e : edges)
    if (d[e.a] < INF)
  if (d[e.b] > d[e.a] + e.cost) {
      d[e.b] = max(-INF, d[e.a] + e.cost);
      p[e.b] = e.a;
      x = e.b;
  }
}
if (x == -1)cout << "No negative cycle from " << v;
else{
int y = x;
for (int i = 0; i < n; ++i)y = p[y];
vector<int> path;
for (int cur = y;; cur = p[cur]) {
    path.push_back(cur);
    if (cur == y && path.size() > 1)break;
}
reverse(path.begin(), path.end());
}
```

3.8 Floyd-Warshall

```cpp
for (int i = 0; i < n; ++i)d[i][i] = 0;

for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

3.9 SCC Kosaraju

```cpp
class SCC{
private:
  int n;
  vector<vi> adj;
  vector<vi> adj_t;
  vector<vi> adj_cond;
  vi order;
  vector<bool> vis;
  vi who;
  int comp;
public:
  SCC(int n,vector<vi> &a,vector<vi>&b):n(n),adj(a),adj_t(b){
    comp = -1;
    who.resize(n);
    vis.resize(n);
    for(int i=0;i<n;i++)
      if(!vis[i])dfs1(i);

    vis.assign(n,0);
    for(int i=n-1;i>=0;i--)
      if(!vis[order[i]]){
        comp++;
        dfs2(order[i]);
      }
    adj_cond.resize(comp+1);
    for(int u=0;u<n;u++)
      for(auto v:adj[u])
        if(who[v]!=who[u])
          adj_cond[who[u]].pb(who[v]);
  }

  void dfs1(int u) {
    vis[u] = true;
    for (int v:adj[u])
      if (!vis[v])dfs1(v);
    order.pb(u);
  }

  void dfs2(int u) {
```

```
    vis[u] = 1;
    for (int v : adj_t[u])
      if (!vis[v])dfs2(v);
    who[u] = comp;
  }
};
```

3.10 Flows

3.10.1 Dinic

```
struct flowEdge{
int u,v;
ll cap,flow = 0;
flowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap) {};
};

struct Dinic{
vector<flowEdge> edges;
vector<vi> adj;
int n,s,t;
int id = 0;
vi level, next;

queue<int> q;

Dinic(int n, int s, int t) : n(n), s(s), t(t) {
  adj.resize(n);
  level.resize(n);
  next.resize(n);
  fill(all(level),-1);
  level[s] = 0;
  q.push(s);
}

void add(int u, int v, ll cap){
  edges.emplace_back(u,v,cap);
  edges.emplace_back(v,u,0);
  adj[u].pb(id++);
  adj[v].pb(id++);
}

bool bfs(){
  while (!q.empty()){
    int curr = q.front();
    q.pop();
    for (auto e : adj[curr]){
      if (edges[e].cap - edges[e].flow  < 1) continue;
      if (level[edges[e].v] != -1) continue;
      level[edges[e].v] = level[edges[e].u] + 1;
      q.push(edges[e].v);
    }
  }
  return level[t] != -1;
}

ll dfs(int u, ll flow){
  if (flow == 0) return 0;
  if (u == t) return flow;
  for (auto& cid = next[u]; cid < adj[u].size(); cid++){
      int e = adj[u][cid];
      int v = edges[e].v;

      if (level[edges[e].u] + 1 != level[v] || edges[e].cap - edges[e].
          flow < 1) continue;
      ll f = dfs(v,min(flow,edges[e].cap - edges[e].flow));
      if (f == 0) continue;
      edges[e].flow += f;
      edges[e ^ 1].flow -= f;
      return f;
  }
  return 0;
}

ll maxFlow(){
  ll flow = 0;
  while (bfs()){
      fill(all(next),0);
      for (ll f = dfs(s,INF64); f != 0ll; f = dfs(s,INF64)) flow += f;
      fill(all(level),-1);
      level[s] = 0;
      q.push(s);
  }
  return flow;
    }
```

```
vector<pii> minCut(){
  vector<pii> ans;
  fill(all(level),-1);
  level[s] = 0;
  q.push(s);
  while (!q.empty()){
    int curr = q.front();
    q.pop();
    for (int id = 0; id < adj[curr].size(); id++){
      int e = adj[curr][id];
      if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow > 0){
        q.push(edges[e].v);
        level[edges[e].v] = level[edges[e].u] + 1;
      }
    }
  }

  for (int i = 0; i < level.size(); i++){
    if (level[i] != - 1){
      for (int id = 0; id < adj[i].size(); id++){
      int e = adj[i][id];
      if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow == 0)
        ans.emplace_back(edges[e].u,edges[e].v);
      }
    }
  }
  return ans;
}

vector<pii> MaximumMatching(){
  vector<pii> ans;
  fill(all(level),-1);
  level[s] = 0;
  q.push(s);
  while (!q.empty()){
    int curr = q.front();
    q.pop();
    for (int id = 0; id < adj[curr].size(); id++){
    int e = adj[curr][id];
    if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow == 0 &&
        edges[e].flow != 0ll){
        q.push(edges[e].v);
        level[edges[e].v] = level[edges[e].u] + 1;
      }
    }
  }
  for (int i = 0; i < level.size(); i++){
    if (level[i] != - 1)
      for (int id = 0; id < adj[i].size(); id++){
        int e = adj[i][id];
        if (edges[e].u != s && edges[e].v != t && edges[e].cap - edges[e
            ].flow == 0 && edges[e].flow != 0ll)
          ans.emplace_back(edges[e].u,edges[e].v);
      }
  }
  return ans;
}

};
```

3.11 2-SAT

```
class SAT{
private:
  int n;
  vector<vi> adj;
  vector<vi> adj_t;
  vector<vi> adj_cond;
  vi order;
  vector<bool> vis;
  vi who;
  vector<bool> ans;
  int comp;
public:

  SAT(int m=0){
    n = (m<<1);
    adj.resize(n);
    adj_t.resize(n);
    vis.assign(n,0);
    who.assign(n,-1);
  }

  void dfs1(int u) {
    vis[u] = true;
    for (int v:adj[u])
```

```
      if (!vis[v])dfs1(v);
    order.pb(u);
  }

  void dfs2(int u) {
    vis[u] = 1;
    for (int v : adj_t[u])
      if (!vis[v])dfs2(v);
    who[u] = comp;
  }

  //~a->b ~b->a
  void addOR(int a, bool af, int b, bool bf) {
    a += a + (af ^ 1);
    b += b + (bf ^ 1);
    adj[a ^ 1].push_back(b);
    adj[b ^ 1].push_back(a);
    adj_t[b].push_back(a ^ 1);
    adj_t[a].push_back(b ^ 1);
  }

  //( a OR b ) ^ (~a or ~b)
  void addXOR(int a, bool af, int b, bool bf) {
    addOR(a, af, b, bf);
    addOR(a, !af, b, !bf);
  }
  //(a -> b)
  void _add(int a,bool af,int b,bool bf) {
    a += a + (af ^ 1);
    b += b + (bf ^ 1);
    adj[a].push_back(b);
    adj_t[b].push_back(a);
  }

  //(a->b)^(b->a)
  void add(int a,bool af,int b,bool bf) {
    _add(a, af, b, bf);
    _add(b, !bf, a, !af);
  }

  bool solve_SAT(){
    for(int i=0;i<n;i++)
      if(!vis[i])dfs1(i);

    comp = -1;
    vis.assign(n,0);
    for(int i=sz(order)-1;i>=0;i--)
      if(!vis[order[i]]){
        comp++;
        dfs2(order[i]);
      }
    ans.assign(n/2,0);
    for(int i=0;i<n;i+=2){
      if(who[i] == who[i+1])return 0;
      ans[i/2] = (who[i] > who[i+1]);
    }
    return 1;
  }

  bool operator[](int i){
    return ans[i];
  }

};
```

3.12 Euler Paths and cycles

3.12.1 Directed Graphs

```
struct EulerSolver{
private:
  vector<vector<pii>> adj;
  int n,edges;
  vi in,out;
  vi ans,edge_ans;
  int root;
  vi done;
  void dfs(int u){
    while(done[u] < sz(adj[u])){
      auto [v,id] = adj[u][done[u]++];
      dfs(v);
      edge_ans.pb(id);
    }
    ans.pb(u);
  }
public:
  EulerSolver(vector<vector<pii>> &a,int t):adj(a){
```

```
    n = t;
    edges = 0;
    in.assign(n,0);
    out.assign(n,0);
    done.assign(n,0);
    root = -1;
    for(int u=0;u<n;u++)
      for(auto v:adj[u]){
        edges++;
        in[v.F]++;
        out[u]++;
      }

  }

  bool possible(){
    int cnt1 =0,cnt2 =0;
    bool ok = 1;
    for(int i=0;i<n;i++){
      if (in[i] - out[i] == 1) cnt1++;
      if (out[i] - in[i] == 1) cnt2++, root = i;
      if (abs(in[i] - out[i]) > 1) ok = 0;
    }
    if(cnt1 > 1 || cnt2 > 1)ok = 0;
    if(!ok)return 0;

    if(root == -1)
      for(int i=0;i<n;i++)
        if(out[i])root = i;

    if(root == -1)return 1;
    dfs(root);

    if(sz(ans) != edges+1)return 0;
    reverse(all(ans));
    reverse(all(edge_ans));
    return 1;
  }

  int size(){
    return ans.size();
  }

  int operator()(int i,bool x){
    if(x)return ans[i];
    else return edge_ans[i];
  }
};
```

3.12.2 Unirected Graphs

```
struct EulerSolver{
private:
  vector<vector<pii>> adj;
  int n,edges;
  vi ans,edge_ans;
  vi deg;
  int root;
  vi done;
  vector<bool> vis;

  void dfs(int u){
    while(done[u] < sz(adj[u])){
      auto [v,id] = adj[u][done[u]++];
      if(vis[id])continue;
      vis[id]=1;
      dfs(v);
      edge_ans.pb(id);
    }
    ans.pb(u);
  }
public:
  EulerSolver(vector<vector<pii>> &a,int t):adj(a),n(t){
    edges = 0;
    done.assign(n,0);
    deg.assign(n,0);
    root = -1;
    for(int u=0;u<n;u++)
      for(auto v:adj[u]){
        edges++;
        deg[v.F]++;
        deg[u]++;
      }
    vis.assign(edges/2,0);
  }

```

```
  bool possible(){
    int odd = 0;
    for(int i=0;i<n;i++)
      if((deg[i]/2)&1){
        odd++;
        root = i;
      }


    if(odd>2)return 0;

    if(root == -1)
      for(int i=0;i<n;i++)
        if(deg[i]){root = i;break;}

    if(root == -1)return 1;
    dfs(root);

    if(sz(ans) != edges/2+1)return 0;
    reverse(all(ans));
    reverse(all(edge_ans));
    return 1;
  }

  int size(){
    return ans.size();
  }

  int operator()(int i,bool x){
    if(x)return ans[i];
    else return edge_ans[i];
  }
};
```

3.12.3 Bridges

```

vector<pii> bridge;
vi disc(MAX),low(MAX);
vector<vi> adj(MAX);
int Time = 0;

void dfs(int u,int p){
  disc[u] = low[u] = ++Time;
  for(int v:adj[u]){
    if(v==p)continue;
    if(!disc[v]){
      dfs(v,u);
      if(disc[u]<low[v])bridge.emplace_back(u,v);
      low[u] = min(low[u],low[v]);
    }else{
      low[u] = min(low[u],disc[v]);
    }

  }
}
```

4 Trees

4.1 LCA - Binary Lifting

4.1.1 Normal LCA

```
const int MAX = 1e5+5,LG=18;
vi deep(MAX);//Si tiene peso vi cost(MAX)
int par[MAX][LG+1];

void dfs(int u = 1,int p=0){// U = raiz del arbol
  par[u][0] = p;
  deep[u] = deep[p]+1;//cost[u] += cost[p]
  for(int i=1;i<=LG;i++)par[u][i]=par[par[u][i-1]][i-1];
  for(int v:adj[u]){
    if(v!=p){
      dfs(v,u);
    }
  }
}

int lca(int u,int v){
  if(deep[u] < deep[v])swap(u,v);
  for(int k=LG;k>=0;k--)
    if(deep[par[u][k]] >= deep[v])
      u = par[u][k];
  if(u==v)return u;
  for(int k=LG;k>=0;k--)
    if(par[u][k]!=par[v][k])
      u=par[u][k],v=par[v][k];
  return par[u][0];
}

int dist(int u,int v){
  int lc = lca(u,v);
  return deep[u]+deep[v]-(deep[lc]<<1);
}

int kth(int u,int k){
  assert(k>=0);
  for(int i=0;i<=LG;i++)
    if(k&(1<<i))u = par[u][i];
  return u;
}

int isanc(int u, int g) {
  int k = dep[u] - dep[g];
  return k >= 0 && kth(u, k) == g;
}
```

4.1.2 Min/Max weight on the path

```
const int MAX = 1e5+5,LG=18;
vector<vector<pii>> adj(MAX,vector<pii> ());
vi deep(MAX);
int par[MAX][LG+1];
int mn[MAX][LG+1];
int mx[MAX][LG+1];

void dfs(int u=1,int p=0,int costmn = INF,int costmx = -INF){
  par[u][0] = p;
  deep[u] = deep[p]+1;

  if(p!=0){
    mn[u][0]  = costmn;
    mx[u][0]  = costmx;
  }
  for(int i=1;i<=LG;i++){
    par[u][i]=par[par[u][i-1]][i-1];
    mn[u][i]=min(mn[u][i-1],mn[par[u][i-1]][i-1]);
    mx[u][i]=max(mx[u][i-1],mx[par[u][i-1]][i-1]);
  }

  for(auto [v,t]:adj[u]){
    if(v!=p){
      dfs(v,u,t,t);
    }
  }
}

pii minmax(int u,int v){
  pii ans = {INF,-INF};
  if(deep[u] < deep[v])swap(u,v);
  for(int k=LG;k>=0;k--)if(deep[par[u][k]] >= deep[v]){
    ans.F = min(ans.F,mn[u][k]);
    ans.S = max(ans.S,mx[u][k]);
```

```
    u = par[u][k];
  }
  if(u==v)return ans;
  for(int k=LG;k>=0;k--)if(par[u][k]!=par[v][k]){
    ans.F = min({ans.F,mn[u][k],mn[v][k]});
    ans.S = max({ans.S,mx[u][k],mx[v][k]});
    u=par[u][k],v=par[v][k];
  }
  ans.F = min({ans.F,mn[u][0],mn[v][0]});
  ans.S = max({ans.S,mx[u][0],mx[v][0]});

  return ans;
}
```

4.1.3 Union path

```
pair<int, int> merge(pair<int, int> x, pair<int, int> y) {
  if (x.first == 0)return y; //where {0,0} is a null path
  if (y.first == 0)return x;
  if (x.first == -1 || y.first == -1)   return { -1, -1};
  vector<int> can = {x.first, x.second, y.first, y.second};
  int a = can[0];
  for (int u : can)
    if (dep[u] > dep[a])
      a = u;
  int b = -1;
  for (int u : can)
    if (!isanc(a, u)) {
      if (b == -1)  b = u;
      if (dep[b] < dep[u])    b = u;
    }
  if (b == -1) {
    b = can[0];
    for (int u : can)
      if (dep[u] < dep[b])
        b = u;
    return {a, b};
  }
  int g = lca(a, b);
  for (int u : can) {
    if (u == a || u == b)continue;
    if (dep[u] < dep[g] || (!isanc(a, u) && !isanc(b, u)))return { -1,
        -1};
  }
  return {a, b};
}
```

4.1.4 Intersection Path

```
pair<int, int> getCommonPath(int u, int a, int v, int b) {
// returns common path between u->a and v->b where u is ancestor a and v
     is ancestor b
  if (!isanc(a, v)) return make_pair(0, 0);
  int x = lca(a, b);
  if (dep[v] < dep[u]) {
    if (isanc(x, u))
      return make_pair(u, x);
  }else{
    if (isanc(x, v))
      return make_pair(v, x); // v->x is the common path
  }
  return make_pair(0, 0);
}
pair<int, int> path_union_light(pair<int, int> x, pair<int, int> y) {
  if (x.first == 0) return y; //where {0,0} is a null path
  if (y.first == 0) return x;
  vector<int> can = {x.first, x.second, y.first, y.second};
  int a = can[0];
  for (int u : can) if (dep[u] > dep[a]) a = u;
  int b = -1;
  for (int u : can)if (!isanc(a, u)) {
    if (b == -1) b = u;
    if (dep[b] < dep[u]) b = u;
  }
  if (b == -1) {
    b = can[0];
    for (int u : can) if (dep[u] < dep[b]) b = u;
    return {a, b};
  }
  return {a, b};
}
// returns the intersection of two paths (a,b) and (c,d)
// {0,0} for null path
pair<int, int> path_intersection(int a, int b, int c, int d) {
  if (a == 0 || b == 0) return {0, 0};
  int u = lca(a, b), v = lca(c, d); // splits both paths in two chains
```

```
      from lca to one node
  pair<int, int> x, y;
  x = getCommonPath(u, a, v, c);
  y = getCommonPath(u, a, v, d);
  pair<int, int> a1 = path_union_light(x, y);
  x = getCommonPath(u, b, v, c);
  y = getCommonPath(u, b, v, d);
  pair<int, int> a2 = path_union_light(x, y);
  return path_union_light(a1, a2);
}
```

4.2 Centroid Decomposition

```
class Centroid{
private:
  vector<vi> tree;
  vi par;
  vi sub;
  vector<bool> check;
  int n;
public:
  Centroid(vector<vi> &t):tree(t){
    n = tree.size();
    par.resize(n);
    sub.resize(n);
    check.resize(n);
    build();
  }

  void build(int u = 0, int p = -1){
    dfs(u, p);
    int c = get_centroid(u,p,sub[u]);
    check[c] = 1;
    par[c] = p+1;

    for (auto v : tree[c]){
      if(!check[v])
        build(v, c);
    }

  }

  int dfs(int u,int p){
    if(check[u])return 0;
    sub[u] = 1;
    for(int v:tree[u])
      if(v!=p)sub[u]+=dfs(v,u);

    return sub[u];
  }

  int get_centroid(int u,int p,int x){
    for(auto v:tree[u])
      if(v!=p && sub[v]*2>x && !check[v])return get_centroid(v,u,x);

    return u;
  }

  int operator[](int i){
    return par[i];
  }
};
```

4.3 Euler Tour Technique

```
int timer = 0;
vi start,final;
vector<vi> adj;
void dfs(int u = 0,int prev = -1){
  start[u]=timer++;
  for(int v:adj[u]){
    if(v!=prev)dfs(v,u);
  }
  final[u] = timer;
}
// [start,final)
```

4.4 Tree Matching

```
vector<vll> dp(2,vll(MAX));

void dfs(ll u = 0, ll p=-1){
  dp[0][u] = 0;
  dp[1][u] = -INF;

  for(ll v:adj[u]){
    if(v == p)continue;
```

```
    dfs(v,u);

    dp[0][u]+=max(dp[1][v],dp[0][v]);

    ll opt = min(dp[0][v]-dp[1][v],0ll);

    dp[1][u] = max(dp[1][u],opt);
  }

  dp[1][u] += dp[0][u];
  dp[1][u]++;
}
```

4.5 Diameter

```
vector<vector<pii>> adj;
vector<bool> vis;

int bfs(int n,vll &d,int v = 0){
  vis.assign(n,0);
  d.assign(n,0);
  d[v] = 0;
  queue<int> q;
  q.push(v);
  pll last = {v,0};
  vis[v] = 1;
  while(!q.empty()){
    int u = q.front();q.pop();
    for(auto [w,c]:adj[u]){
      if(vis[w])continue;
      d[w] = d[u]+c;
      q.push(w);
      vis[w] = 1;
      if(d[w]>last.S)last = {w,d[w]};
    }
  }
  return int(last.F);
}

void solve(){
  int n;cin >> n;
  adj.assign(n,vector<pii> ());
  for(int i=1;i<n;i++){
    int a,b;ll c;
    cin >> a >> b >> c;
    adj[--a].pb({--b,c});
    adj[b].pb({a,c});
  }
  vll dx(n),dy(n);
  int a =bfs(n,dx);
  int b =bfs(n,dy,a);
  bfs(n,dx,b);

  for(int i=0;i<n;i++){
    cout << max(dx[i],dy[i]) << " \n"[i==(n-1)];
  }

}
```

4.6 Heavy Light Decomposition

```
class HLD{
private:
  int n;
  vector<vi> adj;
  vi parent;
  vi heavy;
  vi depth;
  vi root;
  vi treePos;
  Segment_Tree tree;

  int dfs(int u =0){
    int size = 1, mx_sub = 0;
    for(auto v:adj[u])
      if(v!=parent[u]){
        parent[v] = u;
        depth[v] = depth[u]+1;
        int sub = dfs(v);
        if(sub > mx_sub){
          heavy[u] = v;
          mx_sub = sub;
        }
        size += sub;
      }
    return size;
```

```
  }

  template <class BinaryOperation>
  void processPath(int u, int v, BinaryOperation op) {
    for (; root[u] != root[v]; v = parent[root[v]]) {
      if (depth[root[u]] > depth[root[v]]) swap(u, v);
      op(treePos[root[v]], treePos[v] + 1);
    }
    if (depth[u] > depth[v]) swap(u, v);
    op(treePos[u], treePos[v] + 1);
  }

  void init(vi &a){
    for(int i=0,crt=0;i<n;++i){
      if(parent[i] == -1 || heavy[parent[i]]!=i){
        for(int j = i;j!=-1;j = heavy[j]){
          root[j] = i;
          treePos[j] = crt++;
        }
      }
    }
    tree = Segment_Tree(n);
    for(int i=0;i<n;i++){
      tree.update(treePos[i],a[i]);
    }
  }

public:

  HLD(int n,vector<vi> &vale,vi &a):n(n),adj(vale){
    parent.assign(n,0);
    heavy.assign(n,-1);
    depth.assign(n,0);
    root.resize(n);
    treePos.assign(n,0);
    parent[0] = -1;
    dfs();
    init(a);
  }

  void update(int u,int &val){
    tree.update(treePos[u],val);
  }

  void updatePath(int u, int v, int value) {
    processPath(u, v, [this, &value](int l, int r) {
      tree.update_range(l, r, value);
    });
  }

  int query(int u,int v){
    int ans = 0;
    processPath(u,v,[this,&ans](int l,int r){
      ans = max(ans,tree.query(l,r));
    });
    return ans;
  }
};
```

4.7 Prufer Code

4.7.1 Build

```
auto dfs = [&](auto &self,int u){
    for (int v:adj[u]) {
        if (v != parent[u]) {
            parent[v] = u;
            dfs(v);
        }
    }
};

parent[n] = -1;
dfs(dfs,n);

int ptr = -1;
vi degree(n);
for(int i=1;i<=n;i++) {
        degree[i] = sz(adj[i]);
        if(degree[i]==1 && ptr==-1)
    ptr = i;
}

vi code(n - 2);
int leaf = ptr;
for(int i=0;i<n-2;i++){
  int next = parent[leaf];
```

```
  code[i] = next;
  if (--degree[next]==1 && next<ptr) {
    leaf = next;
  }else{
    ptr++;
    while (degree[ptr] != 1)
      ptr++;
    leaf = ptr;
  }
}
```

4.7.2 Restore

```
int n = code.size() + 2;
vi degree(n,0);
for (int i : code)
  degree[i]++;

int ptr = 0;
while (degree[ptr] != 1)
  ptr++;

int leaf = ptr;

vector<pii> edges;
for(int v:code){
  edges.emplace_back(leaf, v);
  if (--degree[v] == 0 && v < ptr) {
    leaf = v;
  }else{
    ptr++;
    while (degree[ptr] != 1)
      ptr++;
    leaf = ptr;
  }
}

edges.emplace_back(leaf, n-1);
```

4.7.3 Cayley's Formula

Cayley's formula states that the number of spanning trees in a complete labeled graph with n vertices is equal to:

$$n^{n-2}$$

In fact any Prufer code with $n-2$ numbers from the interval $[0; n-1]$ corresponds to some tree with n vertices. So we have n^{n-2} different such Prufer codes. Since each such tree is a spanning tree of a complete graph with n vertices, the number of such spanning trees is also n^{n-2}.

4.7.4 Number of ways to make a graph connected

$$s_1 \cdot s_2 \cdots s_k \cdot (s_1 + s_2 + \cdots + s_k)^{k-2} = s_1 \cdot s_2 \cdots s_k \cdot n^{k-2}$$

By accident this formula also holds for $k = 1$.

5 Math

5.1 FFT

```
struct base {
  double a, b;
  base(double a = 0, double b = 0) : a(a), b(b) {}
  const base operator + (const base &c) const{
    return base(a + c.a, b + c.b);
  }
  const base operator - (const base &c) const{
    return base(a - c.a, b - c.b);
  }
  const base operator * (const base &c) const{
    return base(a * c.a - b * c.b, a * c.b + b * c.a);
  }
};
void fft(vector<base> &p, bool inv = 0) {
  int n = p.size(), i = 0;
  for(int j = 1; j < n - 1; ++j) {
    for(int k = n >> 1; k > (i ^= k); k >>= 1);
    if(j < i) swap(p[i], p[j]);
  }

  for(int l = 1, m; (m = l << 1) <= n; l <<= 1) {
    double ang = 2 * PI / m;
    base wn = base(cos(ang), (inv ? 1. : -1.) * sin(ang)), w;
    for(int i = 0, j, k; i < n; i += m) {
      for(w = base(1, 0), j = i, k = i + l; j < k; ++j, w = w * wn) {
        base t = w * p[j + l];
        p[j + l] = p[j] - t;
        p[j] = p[j] + t;
```

```
      }
    }
  }

  if(inv) for(int i = 0; i < n; ++i) p[i].a /= n, p[i].b /= n;
}
vector<ll> multiply(vi &a, vi &b) {
  int n = a.size(), m = b.size(), t = n + m - 1, sz = 1;
  while(sz < t) sz <<= 1;
  vector<base> x(sz), y(sz), z(sz);
  for(int i = 0 ; i < sz; ++i) {
    x[i] = i < (int)a.size() ? base(a[i], 0) : base(0, 0);
    y[i] = i < (int)b.size() ? base(b[i], 0) : base(0, 0);
  }
  fft(x), fft(y);
  for(int i = 0; i < sz; ++i) z[i] = x[i] * y[i];
  fft(z, 1);
  vector<ll> ret(sz);
  for(int i = 0; i < sz; ++i) ret[i] = (long long) round(z[i].a);
  while((int)ret.size() > 1 && ret.back() == 0) ret.pop_back();
  return ret;
}
```

5.1.1 XOR

$$a + b = a \oplus b + 2(a\&b)$$

$$a + b = a|b + a\&b$$

$$a \oplus b = a|b - a\&b$$

$$1 \oplus 2 \oplus 3 \oplus \cdots \oplus (4k - 1) = 0k >= 0$$

5.2 Theorems

6 Strings

6.1 Hashing

6.1.1 Rolling Hashing

```
ll binpow(ll n,ll k,const int mod){
  ll res = 1;
  n%=mod;
  while(k){
    if(k&1)(res*=n)%=mod;
    (n*=n)%=mod;
    k>>=1;
  }
  return res;
}

const int MOD1 = 127657753,MOD2 = 987654319;
const int P1 = 137,P2 = 277;
const int MAX = 1e6+9;

vector<pll> P(MAX),I_P(MAX);

void calc(){
  P[0] = {1,1};
  for(int i=1;i<MAX;i++){
    P[i].F = P[i-1].F*P1%MOD1;
    P[i].S = P[i-1].S*P2%MOD2;
  }
  int i1 = binpow(P1,MOD1-2,MOD1);
  int i2 = binpow(P2,MOD2-2,MOD2);
  I_P[0] = {1,1};
  for(int i=1;i<MAX;i++){
    I_P[i].F = I_P[i-1].F*i1%MOD1;
    I_P[i].S = I_P[i-1].S*i2%MOD2;
  }
}

struct Hashing {
  int n;
  string s; // 0 - indexed
  vector<pll> hs; // 1 - indexed

  Hashing(string _s) {
    n = _s.size();
    s = _s;
    hs.emplace_back(0, 0);
    for (int i = 0; i < n; i++) {
      pll p;
      p.F = (hs[i].F + P[i].F * (s[i]-'a'+1) % MOD1) % MOD1;
      p.S = (hs[i].S + P[i].S * (s[i]-'a'+1) % MOD2) % MOD2;
      hs.push_back(p);
    }
```

```
  }

  pll get_hash(int l, int r) { // 1 - indexed
    pll ans;
    ans.F = (hs[r].F - hs[l - 1].F+ MOD1) * 1LL * I_P[l - 1].F% MOD1;
    ans.S = (hs[r].S - hs[l - 1].S+ MOD2) * 1LL * I_P[l - 1].S% MOD2;
    return ans;
  }
};
```

6.2 Suffix Array

```
struct suffix{
  int index;
  pii rank;
  bool operator<(suffix b){return rank<b.rank;}
  bool operator>(suffix b){return rank>b.rank;}
};

class Suffix_Array{
  public:
  string s;int n;
  vector<suffix> suffixes;
  vi lcp;
  Suffix_Array(string x){
    s = x+"$";
    n = sz(s);
    suffixes.resize(n);
  }


  void radix_sort(vector<suffix> &suff){
    for(int i: vi{2,1}){
      auto key = [&](const suffix &x){
        return i == 1?x.rank.F:x.rank.S;
      };

      int mx = 0;
      for (const auto &i:suff) { mx = max(mx, key(i)); }
      vector<int> occs(mx + 1);
      for (const auto &i:suff)occs[key(i)]++;
      vector<int> start(mx + 1);
      for (int i=1;i<=mx; i++) start[i] = start[i-1]+occs[i-1];

      vector<suffix> new_arr(suff.size());
      for (const auto &i : suff) {
        new_arr[start[key(i)]] = i;
        start[key(i)]++;
      }
      suff = new_arr;
    }
  }

  void build(){
    for(int i=0;i<n;i++)suffixes[i].index = i , suffixes[i].rank = {s[i
        ],(i+1<n?s[i+1]:-1)};
    sort(all(suffixes));
    vi equiv(n);

    for(int i=1;i<n;i++){
      auto c_val = suffixes[i].rank;
      int cs = suffixes[i].index;
      auto  p_val = suffixes[i - 1].rank;
      int ps = suffixes[i-1].index;
      equiv[cs] = equiv[ps] + (c_val > p_val);
    }

    for(int cmp_ant = 1;cmp_ant <n; cmp_ant<<=1){
      for(auto &x:suffixes)
        x.rank = {equiv[x.index],equiv[(x.index+cmp_ant)%n]};

      //Std sort O(nlognlogn) Radix Sort O(nlogn)
      radix_sort(suffixes);

      for(int i=1;i<n;i++){
        auto c_val = suffixes[i].rank;
        int cs = suffixes[i].index;
        auto  p_val = suffixes[i - 1].rank;
        int ps = suffixes[i-1].index;
        equiv[cs] = equiv[ps] + (c_val > p_val);
      }
    }
  }

  void build_lcp(){
    vector<int> suff_ind(n);
```

```
    for (int i = 0; i < n; i++)suff_ind[suffixes[i].index] = i;
    lcp.resize(n-1);
    int start_at = 0;
    for (int i = 0; i < n - 1; i++) {
      int prev = suffixes[suff_ind[i] - 1].index;
      int curr_cmp = start_at;
      while (s[i + curr_cmp] == s[prev + curr_cmp])curr_cmp++;
      lcp[suff_ind[i] - 1] = curr_cmp;
      start_at = max(curr_cmp - 1, 0);
    }
  }

  void show(){
    for(int i=0;i<n;i++)cout << suffixes[i].index << " \n"[(i+1)==n];
  }
};
```

7 Others

7.1 Binpow

```
ll binpow(ll a,ll b){
  a%=MOD;
  ll ans = 1;
  while(b){
    if(b&1)(ans*=a)%=MOD;
    (a*=a)%=MOD;
    b>>=1;
  }
  return ans;
}
```

7.2 Matrix exponentiation

```
struct Mat {
  int n, m;
  vector<vector<int>> a;
  Mat() { }
  Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
  Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0;
      a = v; }
  inline void make_unit() {
    assert(n == m);
    for (int i = 0; i < n; i++)  {
      for (int j = 0; j < n; j++) a[i][j] = i == j;
    }
  }
  inline Mat operator + (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
      }
    }
    return ans;
  }
  inline Mat operator - (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
      }
    }
    return ans;
  }
  inline Mat operator * (const Mat &b) {
    assert(m == b.n);
    Mat ans = Mat(n, b.m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < b.m; j++) {
        for(int k = 0; k < m; k++) {
          ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] % mod)
              % mod;
        }
      }
    }
    return ans;
  }
  inline Mat pow(long long k) {
    assert(n == m);
    Mat ans(n, n), t = a; ans.make_unit();
    while (k) {
      if (k & 1) ans = ans * t;
      t = t * t;
```

```
      k >>= 1;
    }
    return ans;
  }
  inline Mat& operator += (const Mat& b) { return *this = (*this) + b; }
  inline Mat& operator -= (const Mat& b) { return *this = (*this) - b; }
  inline Mat& operator *= (const Mat& b) { return *this = (*this) * b; }
  inline bool operator == (const Mat& b) { return a == b.a; }
  inline bool operator != (const Mat& b) { return a != b.a; }
};

// Usage
// Mat a(n, n);
// Mat b(n, n);
// Mat c = a * b;
// Mat d = a + b;
// Mat e = a - b;
// Mat f = a.pow(k);
// a.a[i][j] = x;
```

7.3 Random

```
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
```